

# Introduction to Cassandra Query Language Shell (cqlsh)

Sure! Let's dive into the concept of the Cassandra Query Language (CQL) and its special commands.

CQL is like a friendly language that helps you talk to a database called Apache Cassandra. Imagine you have a big library (the database) filled with books (data), and CQL is the way you ask the librarian (Cassandra) to find, add, or change books. Just like how you might say, "Can I have the book on gardening?" in a library, you use CQL to say things like "Show me all the users" or "Add a new user." It has a simple structure that feels a bit like SQL, which is another language for databases, but there are some differences. For example, in CQL, you can't ask for a "join" of two tables like you can in SQL; instead, you organize your data in a way that makes it easy to find without needing to join.

Now, CQL has some special commands that make it even more powerful. For instance, the "COPY" command lets you easily move data in and out of your database, like taking a book out of the library or returning it. Another important command is "CONSISTENCY," which helps you decide how many copies of your data need to agree before you can say an operation is successful. Think of it like asking a group of friends for their opinions: if most of them agree, you can trust that it's a good idea!

# Cassandra Query Language

---

- CQL is the primary language for communication with Cassandra clusters
- Simple yet intuitive syntax (SQL-like)
- CQL lacks grammar for relational features such as JOIN statements
- Different behavior of CQL commands vs. SQL

```
• CREATE KEYSPACE intro_cassandra WITH ..  
• CREATE TABLE test () ..  
• INSERT INTO test () VALUES () ..  
• SELECT * FROM test WHERE ..  
• UPDATE test SET age = 25 WHERE userid=30 ..  
• DELETE FROM test WHERE userid=30 ..  
• DROP TABLE test;  
• TRUNCATE TABLE test;
```

- Cassandra Query Language, referred to as CQL, is the primary language for communicating with Apache Cassandra clusters. CQL has a simple and intuitive, SQL-like syntax that allows the creation of keyspaces, tables, inserts, updates, deletes, and select queries.
- While CQL syntax has similarities to SQL, there are many differences as well between what you can do in CQL vs. SQL. One of the most notable differences is that CQL does not support JOIN statements. In Cassandra, if you need a join, then you store the data already joined.
- Also, although, syntax-wise, some operations seem similar to SQL, their behavior in Cassandra is different; for example, inserts, updates, and deletes are done in memory directly, without prior reads to locate the data.

## Cassandra Query Language

---

- CQL keywords are case-insensitive
- Identifiers in CQL are case-insensitive unless enclosed in double quotation marks
- Names for identifiers created using uppercase are stored in lowercase
- Commented text (//) is ignored by CQL

```
Select * from users;  
SELECT * from users;  
SELECT * FROM USERS;  
//all commands are similar
```

```
CREATE TABLE USERS(..);  
CREATE table users(..);  
//Stored name of the table: users
```

- You will see, as you go through the lab exercises, that CQL keywords are case insensitive. Identifiers are also case insensitive, unless they are enclosed in double quotation marks. For example, 'Select' in sentence case, and 'SELECT' in uppercase are treated the same. If you enter names for identifiers, for example a table name, using uppercase letters, Cassandra will just store the names in lowercase anyway. As you can see in the examples, commented text, denoted by a double forward slash, will be ignored by CQL

## Running CQL queries

---

- Run using Cassandra client drivers
  - Java, Python, Ruby, Nodejs, PHP, Scala, Clojure ...
  - Default = open source Datastax Java Driver
- Run using the cqlsh client
  - Python-based command line shell for interacting with Cassandra through CQL
  - Shipped with every Cassandra package
  - Connects to a single node (default node or one specified on the command line)
- Other CQL client editors are available

- To run CQL queries, you can do one of the following: Run them programmatically using a licensed Cassandra client driver. There are many options available, including Java, Python, and Scala. The default driver is the open source Datastax Java Driver.
- Alternatively, you can run them on the CQL Shell client (referred to as 'cqlsh') that is provided with the Cassandra package. CQLSH is a python-based command line shell that uses CQL for communicating with a Cassandra cluster. The software is shipped with every Cassandra package. CQLSH connects to a single node – either the one that the command is run on (default), or the one specified as an option in the command line. There are also other CQL editors on the market that communicate with Cassandra using CQL

## CQL shell (cqlsh)

Using cqlsh, you can:

- Create, alter, drop keyspaces
- Create, alter, drop tables
- Insert, update, delete data
- Execute read queries (SELECT)

```
[cqlsh> use intro_cassandra;;
```

```
[cqlsh:intro_cassandra> select * from users;
```

| username         | age | occupation    |
|------------------|-----|---------------|
| Daveg@gmail.com  | 43  | engineer      |
| JayZ@yahoo.com   | 46  | entertainment |
| Elaine@yahoo.com | 60  | producer      |
| Johns@gmail.com  | 60  | actor         |
| Alaind@gmail.com | 32  | actor         |
| Peterd@gmail.com | 32  | producer      |
| Moirad@yahoo.com | 35  | dj            |

(7 rows)

- Using CQL shell you can: Create, alter, and drop keyspaces Create, alter, and drop tables Insert, update, and delete data Execute queries using SELECT

## Command-line options for cqlsh

```
> cqlsh [options] [host [port]]
```

Options:

- help shows help about the cqlsh command options
- version shows the version of cqlsh being used
- u -user specifies the username to authenticate to Cassandra with
- p -password specifies the password to authenticate to Cassandra with
- k -keyspace specifies a keyspace to authenticate to
- f -file enables execution of commands from a given file
- request-timeout specifies the request timeout in seconds (defaults to 10s)

- But before we look at some cqlsh examples, when launching cqlsh, there are a number of command line options you should know about. Let's just mention a few of them here: help – shows help about the options of CQLSH  
commands version – in order to check the cqlsh version being used user and password – for authentication keyspace – keyspace to authenticate to file – enables execution of commands from a given file. Request timeout – you can set a timeout for your queries, the default being 10 seconds

## An example of cqlsh

Commands used in cqlsh:

```
USE intro_cassandra
```

```
SELECT * FROM groups where groupid=12;
```

```
INSERT INTO groups(groupid,username,group_name,age) VALUES  
(12,'aland@gmail.com','baking',32);
```

```
SELECT * FROM groups;
```

## An example of cqlsh

```
[cqlsh> use intro_cassandra;;
```

```
[cqlsh:intro_cassandra> select * from groups where groupid=12;
```

| groupid | username         | group_name | age |
|---------|------------------|------------|-----|
| 12      | Elaine@yahoo.com | Baking     | 46  |
| 12      | Peterd@gmail.com | Baking     | 32  |

(2 rows)

```
cqlsh:intro_cassandra> insert into groups (groupid, username, group_name, age) values (12, 'Aland@gmail.com', 'Baking',32) ; cqlsh:intro_cassandra> select * from groups where groupid=12;
```

| groupid | username         | group_name | age |
|---------|------------------|------------|-----|
| 12      | Aland@gmail.com  | Baking     | 32  |
| 12      | Elaine@yahoo.com | Baking     | 46  |
| 12      | Peterd@gmail.com | Baking     | 32  |

(3 rows)

```
Commands used in cqlsh:
USE intro_cassandra
SELECT * FROM groups where groupid=12;
INSERT INTO groups(groupid,username,group_name,age) VALUES
(12,'aland@gmail.com','baking',32);
SELECT * FROM groups;
```

- Let's see the cqlsh output for a simple example: Here we're just using the 'intro\_cassandra' keyspace, and we select only the users from groupid 12.
- Then we insert a new record into the groups table for groupid 12, and finally select only the users from groupid 12, and also view the inserted data. And this would be the resulting output.

## cqlsh special commands

**CAPTURE** - Captures the output of a command and adds it to a file

**CONSISTENCY** - Shows the current consistency level and sets a new one

**COPY** - Copies data to and from Cassandra

**DESCRIBE** - Describes the current cluster of Cassandra and its objects

**EXIT** - Terminates the cqlsh session

**PAGING** - Enables or disables paging of the query results

**TRACING** - Enables or disables request tracing

- CQLSH also has some special commands, so let's take a quick look at some of them: CAPTURE – Captures the output of a command and adds it to a file. CONSISTENCY – Shows the current consistency level and sets a new one. COPY – Copies data to and from Cassandra. DESCRIBE – Describes the current cluster of Cassandra and its objects. EXIT – Terminates the cqlsh session. PAGING – Enables or disables the paging of query results.
- TRACING – Enables or disables request tracing. While in your hands-on sessions you will get to experience some of these.

## cqlsh CONSISTENCY

---

- Cassandra is tunable consistent
- Sets the **consistency level** for the operations to follow
- **Consistency** refers to the number of nodes (out of the total replicas) that should respond to a query (write/read) to consider the query successful

```
cqlsh: intro_cassandra_keyspace> CONSISTENCY

ONE
TWO
THREE
QUORUM - majority of nodes from the entire cluster
ALL
LOCAL_QUORUM - majority of nodes from the local data center
              (according to the specified data center set replication for the keyspace)
```

- We will now look in a bit more detail at the CONSISTENCY and COPY commands.
- Let's take a detailed look at the CONSISTENCY command.
- Cassandra is tunable consistent. But what does this mean? Well, it means you have the ability to set Cassandra consistency at an operations level. Consistency in Cassandra means the number of nodes (out of the total replicas) that need to answer a request (write/read) in order for the operation to be considered successful.



- Amongst the options available for the Consistency command are: ONE, TWO, or THREE nodes, QUORUM — which is a majority of nodes out of all replicas in the cluster, ALL replicas, and finally LOCAL\_QUORUM — which is a majority of nodes from the local data center (according to the specified data center set replication for the keyspace).

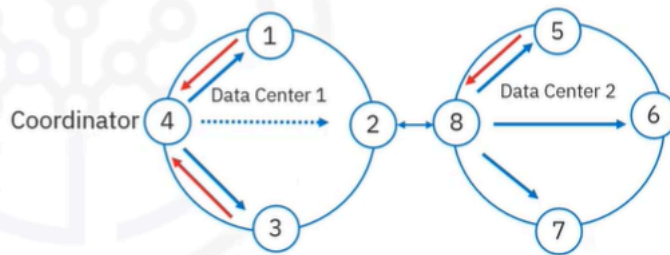
## Consistency example - QUORUM

```
cqlsh: intro_cassandra_keyspace> CONSISTENCY QUORUM
cqlsh: intro_cassandra_keyspace> INSERT INTO groups (groupid,
group_name,username,age) VALUES (45, 'Grilling', 'JayZ@yahoo.com', 46);
```

Multi DC cluster of 8 nodes

DC1 RF 2, DC2 RF 3

- Set consistency QUORUM (3)
- Write operations go to all replicas (5)
- Minimum of 3 nodes (from both DCs) need to answer for a successful operation



- So, for example, if we have a Cassandra cluster of 8 nodes and 2 Data Centers, one data center with replication factor 2 and the other one with replication factor 3, then overall, replication is 5. If we set CONSISTENCY QUORUM on a write operation: first of all, QUORUM means a majority of nodes out of the replicas, so a majority here would be 3. Write operations will go to all 5 replicas but a minimum 3 nodes (from both data centers) out of the 5 replicas need to answer in order for the operation to be successful.



## cqlsh COPY (import/export data)

cqlsh: intro\_cassandra\_keyspace> COPY

CQL shell commands that import and export CSV (comma-separated values or delimited text files). Not suitable for bulk loading.

- **COPY TO** exports data from a table into a CSV file. Each row is written to a line in the target file with fields separated by the delimiter.
- **COPY FROM** imports data from a CSV file into an existing table.
  - Each line in the source file is imported as a row.
  - All rows in the data set must contain the same number of fields and have values in the PRIMARY KEY fields.
  - The process verifies the PRIMARY KEY and imports data accordingly.

- Another important command in cqlsh is the COPY command. While bulk copy in Cassandra should be done through special procedures, if you would like to just test your model or you're working with a smaller, text delimited dataset, then you can use COPY FROM or COPY TO operations to bring data in, and export data out, of Cassandra.
- COPY TO exports data from a table into a CSV file. Each row is written to a line in the target file with fields separated by the delimiter.
- COPY FROM imports data from a CSV file into an existing table. Each line in the source file is imported as a row.
- All rows in the dataset must contain the same number of fields and have values in the PRIMARY KEY fields. The process verifies the PRIMARY KEY and imports data accordingly.

# Recap

---

In this video, you learned that:

- CQL is the primary language for communicating with Apache Cassandra clusters
- CQL keywords and identifiers are case-insensitive
- CQL queries can be run programmatically using a licensed Cassandra client driver, or they can be run on the Python-based CQL shell client provided with Cassandra
- Using the CQL shell, you can create, alter, and drop keyspaces and tables, insert, update, and delete data, and execute queries using SELECT
- CQL shell has several special commands, including the CONSISTENCY and COPY commands
- The CONSISTENCY command can be used to tune the consistency of your data
- The COPY command can be used to import data into and export data out of Cassandra